8f. Type Stability with Tuples

Martin Alfaro
PhD in Economics

INTRODUCTION

A function is type stable when, given its arguments' types, the compiler can deterministically infer a single concrete type for each expression involved. This definition, while universal, takes on different forms when applied to specific objects. So far, we've exclusively dealt with scalars and vectors, whose conditions for type stability are relatively straightforward.

In this section, we expand our analysis of type stability to other data structures. In particular, we consider **tuples**, whose coverage automatically encompasses **named tuples**. Guaranteeing type stability with tuples is more nuanced compared to vectors, as their type characterization demands more information. In fact, their exploration will challenge our understanding of type stability, demanding a clear grasp of their definition and subtleties.

> ⚠️ **Warning!** - Tuples Are Only Suitable For Small Collections
> Remember that tuples should be reserved for collections that **comprise a few elements**. Using them for large collections will result in significant performance degradation or directly trigger fatal errors.

COMPARING TUPLES AND VECTORS

Tuples and vectors are the most common forms of collections in Julia. While both seem to fulfill a similar purpose at the surface, they actually differ significantly in design philosophy and performance characteristics.

In particular, tuples are engineered for objects comprising a small fixed number of elements. When this is the case, tuples outperform vectors by avoiding the memory-allocation overhead incurred by vectors. This advantage will be **explained in more depth later**.

Another key distinction is that tuples possess a more intricate type system relative to vectors. To see this, let's compare the information needed to describe each type.

Vectors represent collections of elements sharing a *homogeneous* type and exhibiting a variable size. Thus, the information needed to describe the types of vectors is relatively minor: simply specifying the element type is sufficient. For instance, a type like `Vector{Float64}` establishes that *all* elements must have type `Float64`, without any restriction on the number of elements to be contained.

For their part, tuples are fixed-size collections that can accommodate *heterogeneous* types. This makes the characterization of its type more demanding, requiring both the number of elements and the type of *each* element. For instance, the variable `tup = ("hello", 1)` has type `Tuple{String, Int64}`, indicating that the first element has type `String` and the second one `Int64`. Furthermore, it implicitly sets the number of elements to two, as there's no possibility of appending or removing elements.

The fact that the element count is intrinsic to the type becomes evident when tuples contain `N` elements of the same type `T`. In such cases, Julia provides the convenient alias `Ntuple{N,T}`. This is just syntactic sugar for `Tuple{T, T,...,T}`, where `T` appears `N` times. [1]

In the following, we show that the choice between tuples and vectors may have different implications for type stability.

TUPLE SLICES WITH MIXED TYPES CAN STILL BE TYPE STABLE

As we indicated, a fundamental distinction between tuples and vectors in Julia lies in how they manage type information. While tuples explicitly define the type of each individual element, vectors require all elements to be of a uniform type.

Because vectors must maintain a consistent type, attempting to store mixed concrete types within a single vector compels Julia to determine a common type that accommodates them all. For example, if you create a vector containing both `Int64` and `Float64`, Julia will infer the type of the vector as `Vector{Float64}`, as it's the most general type encompassing both integers and floats.

However, this automatic widening can lead to significant performance penalties when dealing with highly diverse types. In extreme cases, Julia might resort to using the abstract type `Any`, resulting in a `Vector{Any}`. Working with such vectors is extremely undesirable from a performance standpoint.

This issue also affects vector slices, as they inherit the type information from their parent vector. Thus, if the parent vector has been widened to a more general type like `Vector{Any}`, operations performed on those slices will also be subject to that same type instability. This behavior contrasts sharply with **slices of tuples, where each element within the slice retains its concrete type.**

TUPLE

```
tup    = (1, 2, "hello")        # type is `Tuple{Int64, Int64, String}`

foo(x) = sum(x[1:2])

@code_warntype foo(tup)         # type stable (output is `Int64`)
```

VECTOR

```
vector = [1, 2, "hello"]        # type is `Vector{Any}`

foo(x) = sum(x[1:2])

@code_warntype foo(vector)      # type UNSTABLE
```

TUPLES CONTAIN MORE INFORMATION THAN VECTORS

Given the differences in type information, conversions between tuples and vectors can pose several challenges for type stability.

To see this, let's start with the simplest conversion, where a tuple is transformed into a vector. As our previous analysis suggests, type stability is preserved with this conversion only if all elements within the tuple share the same type or can be promoted to a common concrete type.

To illustrate, recall that each type automatically defines a constructor. This is a specialized function that transforms variables into a corresponding type. For instance, `Vector` can be used as a function that converts variables to the type `Vector`. When you invoke this constructor with a tuple argument, Julia must determine a single element type, capable of holding all items from that tuple. If the tuple contains heterogeneous types that can't be promoted (e.g., mixing strings and integers), the constructor fails to produce a concrete type during the conversion.

TYPE-HOMOGENEOUS TUPLES

```
tup = (1, 2, 3)                 # `Tuple{Int64, Int64, Int64}` or just `NTuple{3, Int64}`


function foo(tup)
    x = Vector(tup)             # 'x' has type `Vector(Int64)}`
    sum(x)
end

@code_warntype foo(tup)         # type stable
```

TYPE PROMOTION

```
tup = (1, 2, 3.5)               # `Tuple{Int64, Int64, Float64}`


function foo(tup)
    x = Vector(tup)             # 'x' has type `Vector(Float64)}`
    sum(x)
end

@code_warntype foo(tup)         # type stable
```

TYPE-HETEROGENEOUS TUPLES

```
tup = (1, 2, "hello")           # `Tuple{Int64, Int64, String}`


function foo(tup)
    x = Vector(tup)             # 'x' has type `Vector(Any)}`
    sum(x)
end

@code_warntype foo(tup)         # type UNSTABLE
```

On the contrary, **constructing a tuple from a vector may cause type instability, even when the vector's type element is concrete**. The core issue is that vectors don't encode their length in their type. As a result, when a vector builds a tuple within a function, the compiler must consider every possible vector length as a distinct tuple type. This forces the generation of code that can handle an open-ended set of type possibilities, thus preventing specialization.

To pass the number of elements, the information must be hard-coded within the function. In other words, you must specify the information as a literal value, rather than as the output of a calculation. In fact, simply passing the count as a function argument won't address the problem, since functions only identify types, not values.

To illustrate, we begin with the case where the vector comprises elements holding an abstract type. In this case, we can't achieve type stability, regardless of how the tuple's length is handled.

TYPE-ELEMENT ABSTRACT

```
x   = [1, 2, "hello"]     # 'Vector{Any}' has no info on each individual type or number of
elements

function foo(x)
     tup    = Tuple(x)        # Tuple with type elements `Any` and undefined number of
arguments



    sum(tup)
end

@code_warntype foo(x)     # type UNSTABLE
```

TYPE-ELEMENT ABSTRACT

```
x   = [1, 2, "hello"]     # 'Vector{Any}' has no info on each individual type or number of
elements

function foo(x, limit)
     tup    = Tuple(x)        # Tuple with type elements `Any` and undefined number of
arguments
    slice = tup[1:limit]  # compiler won't identify number of elements through `limit`

    sum(slice)
end

@code_warntype foo(x,2)   # type UNSTABLE
```

TYPE-ELEMENT ABSTRACT

```
x    = [1, 2, "hello"]      # 'Vector{Any}' has no info on each individual type or number of
elements

function foo(x)
      tup    = Tuple(x)         # Tuple with type elements `Any` and undefined number of
arguments
     slice = tup[1:2]       # `Tuple{Any,Any}`, compiler smart enough to identify number of
elements

    sum(slice)
end

@code_warntype foo(x)     # type UNSTABLE (still operating with `Any`)
```

Instead, when the vector comprises elements holding a concrete type, we can achieve type stability if the number of elements is hard-coded in the function.

TYPE-ELEMENT CONCRETE

```
x    = [1 ,2 ,3]            # 'Vector{Int64}' identifies `Int64`, but no info on the number
of elements

function foo(x)
     tup    = Tuple(x)        # type `Tuple{Vararg{Int64}}` (`Vararg` is "variable number of
arguments")


    sum(tup)
end

@code_warntype foo(x)      # type UNSTABLE
```

TYPE-ELEMENT CONCRETE

```
x    = [1 ,2 ,3]            # 'Vector{Int64}' identifies `Int64`, but no info on the number
of elements

function foo(x,limit)
     tup    = Tuple(x)        # type `Tuple{Vararg{Int64}}` (`Vararg` is "variable number of
arguments")
    slice = tup[1:limit]   # compiler won't identify the number of elements through `limit`

    sum(tup)
end

@code_warntype foo(x,2)    # type UNSTABLE
```

TYPE-ELEMENT CONCRETE

```
x   = [1 ,2 ,3]             # 'Vector{Int64}' identifies `Int64`, but no info on the number
of elements

function foo(x)
    tup   = Tuple(x)        # type `Tuple{Vararg(Int64)}` (`Vararg` is "variable number of
arguments")
    slice = tup[1:2]       # compiler smart enough to identify number of elements

    sum(slice)
end

@code_warntype foo(x)      # type STABLE (the sum)
```

ADDRESSING VARIABLE ARGUMENTS

We indicated that, unless we identify the number of arguments, defining tuples from vectors will inevitably introduce type instability. The most straightforward and effective solution is to form the tuple prior to calling the function, and then pass it as a function argument. Given type inference, this ensures that the compiler identifies all the necessary information for handling tuples. This is demonstrated below.

TUPLE AS A FUNCTION ARGUMENT

```
x   = [1, 2, 3]
tup = Tuple(x)

foo(tup) = sum(tup[1:2])

@code_warntype foo(tup)         # type stable
```

The approach presented should be your first option when transforming vectors to tuples. Nonetheless, there may be scenarios where defining the tuple inside the function is unavoidable. In such cases, there are a few alternatives.

One solution is to define the tuple's length using a literal value. Below, we illustrate this approach. We also show that simply passing the vector's number of elements as a function argument doesn't solve the issue. The reason is that the compiler generates method instances based on information about types, not values. Consequently, a function argument like `length(x)` merely informs the compiler that the number of elements can be described as an object with type `Int64`, without providing any additional insight.

NOT A SOLUTION

```
x   = [1, 2, 3]

function foo(x)
    tup = NTuple{length(x), eltype(x)}(x)

    sum(tup)
end

@code_warntype foo(x)       # type UNSTABLE
```

INFLEXIBLE SOLUTION

```
x   = [1, 2, 3]

function foo(x)
    tup = NTuple{3, eltype(x)}(x)

    sum(tup)
end

@code_warntype foo(tup)     # type stable
```

The downside of this solution is that it defeats the purpose of having generic code: it sacrifices the generality of a function, restricting it to accept only tuples of a single predetermined size. To eliminate the type instability without constraining functionality, we need to introduce a more advanced solution. This is based on a technique known as **dispatch on value**. Because this approach is more complex to implement, *I recommend using it only when passing the tuple as a function argument is unfeasible*.

Next, we begin by laying out the general principles of dispatch on value. Only then we apply the technique to the specific case of tuples.

DISPATCHING ON VALUES

Dispatching on values enables passing information about specific values to the compiler. Since the compiler gathers information about types but not values, implementing this feature requires a workaround: introducing a generic type where the value itself becomes a type parameter. For example, in the case of tuples, this type parameter is simply the vector's number of elements.

The functionality is implemented via the built-in type `Val`, whose type parameter represents the value to be conveyed. Its use is best explained through an example. Suppose a function `foo` that behaves differently depending on a value `a`. The technique requires defining `foo` with a type-annotated argument `::Val{a}`. Technically, this acts as a **constructor**. Since the value of this argument is irrelevant, it's common to leave this argument unnamed and write it directly as `::Val{a}`. Then, when calling the function `foo`, we pass `Val(a)`, where `a` is the value we want the compiler to know.

To illustrate the use of `Val` beyond tuples, suppose that a variable `y` could be an `Int64` or `Float64` contingent upon a condition. Since the compiler can't predict the result of the condition, it can't know which branch will be executed. Consequently, any operation depending on `y` will be type unstable. The strategy is to lift the condition into the type domain using `Val{condition}`. In this way, we create separate methods for `Val{true}` and `Val{false}`, enabling the compiler to specialize each branch. The code snippet below demonstrates this approach in practice.

TYPE UNSTABLE

```
function foo(condition)
    y = condition ? 1 : 0.5      # either `Int64` or `Float64`

    [y * i for i in 1:100]
end

@code_warntype foo(true)         # type UNSTABLE
@code_warntype foo(false)        # type UNSTABLE
```

SOLUTION VIA "VAL"

```
function foo(::Val{condition}) where condition
    y = condition ? 1 : 0.5      # either `Int64` or `Float64`

    [y * i for i in 1:100]
end

@code_warntype foo(Val(true))    # type stable
@code_warntype foo(Val(false))   # type stable
```

> ⚠ **Warning!**
>
> The function argument `Val` must be defined with `{}`, as types define their parameters with `{}`. Instead, `Val` must be called with `()`, as with any other constructor (recall that constructors are ultimately functions).

DISPATCHING ON VALUE WITH TUPLES

Let's now revisit the conversion of vectors to tuples. As we previously discussed, type instability in those cases arises because vectors don't encode their size into their type, leaving the compiler without sufficient information to determine the tuple's type.

Dispatch on value provides a solution to this issue: by passing the vector's length as a type parameter, the function call becomes type stable.

TYPE UNSTABLE

```
x = [1, 2, 3]

function foo(x, N)
    tuple_x = NTuple{N, eltype(x)}(x)

    2 .+ tuple_x
end

@code_warntype foo(x, length(x))        # type UNSTABLE
```

SOLUTION VIA "VAL"

```
x = [1, 2, 3]

function foo(x, ::Val{N}) where N
    tuple_x = NTuple{N, eltype(x)}(x)

    2 .+ tuple_x
end

@code_warntype foo(x, Val(length(x)))   # type stable
```

FOOTNOTES

[1] Don't confuse `NTuple` with an abbreviation for the type `NamedTuple`. The "N" in the former case refers to the number of elements.